CPSC 250
Classes Program Example: The GVDie Craps Game

Edward Brash

1 Program Goal

In this example, we write a program to play an automated dice game using a provided `GVDie` class.

The player rolls two dice. Depending on the result, the player may:

- win one credit,
- lose one credit,
- or set a goal for future rolls.

A round ends when the player either wins or loses a credit. The entire game ends when the player reaches zero credits.

2 The Provided `GVDie` Class

The provided die class represents a single six-sided die.

A simplified version is:

```
import random

class GVDie:

    def __init__(self):
        self.value = None

    def roll(self):
        self.value = random.randint(1, 6)

    def get_value(self):
        return self.value

    def compare_to(self, d):
        return self.value == d.value
```

3 The Meaning of `None`

The constructor initializes:

```
self.value = None
```

The value `None` means that the die does not yet have a meaningful value.

This is useful because a die should not really have a value until it has been rolled.

4 Importing and Using the Die Class

If the class is stored in a file called `gv_die.py`, then the main program can import it:

```
import random
from gv_die import GVDie
```

Then we can create die objects:

```
die1 = GVDie()
die2 = GVDie()
```

Each die object has its own internal value.

5 Random Seeds

The program may ask the user for a random seed:

```
seed = int(input("Enter seed: "))
random.seed(seed)
```

A seed defines the particular sequence of random numbers.
This is useful for testing because it makes program behavior repeatable.

6 Credits

The user also enters the number of credits:

```
credits = int(input("Enter starting credits: "))
```

These credits can be interpreted as the player's number of lives.
The game continues until:

```
credits == 0
```

7 Basic Game Setup

The notes outline the beginning of the program as follows:

1. Create two `GVDie` objects.
2. Initialize the number of rounds.
3. Initialize the goal value to `-1`.
4. Begin the main game loop.

```
die1 = GVDie()
die2 = GVDie()

rounds = 0
goal = -1

while credits > 0:
    ...
```

8 How a Round Works

A round begins with an initial roll of both dice.

```
die1.roll()
die2.roll()

roll = die1.get_value() + die2.get_value()
```

The total determines what happens next.

9 Initial Roll Outcomes

For the initial roll:

- rolling 7 or 11 wins one credit,
- rolling 3 or 12 loses one credit,
- any other value becomes the goal.

```
if roll == 7 or roll == 11:
    credits = credits + 1

elif roll == 3 or roll == 12:
    credits = credits - 1

else:
    goal = roll
```

10 Goal Rolls

If the initial roll sets a goal, the player continues rolling.

On later rolls:

- rolling 7 loses one credit,
- rolling the goal wins one credit,
- any other roll means roll again.

```
while goal != -1:

    die1.roll()
    die2.roll()

    roll = die1.get_value() + die2.get_value()

    if roll == 7:
        credits = credits - 1
        goal = -1

    elif roll == goal:
        credits = credits + 1
        goal = -1
```

11 Why This Is a Class Example

This program is useful because it shows objects in action.

Instead of treating dice as raw integers, we use die objects that know how to:

- roll themselves,
- store their current value,
- return their value,
- compare themselves to another die.

The class bundles together both data and behavior.

12 Complete Program Skeleton

```
import random
from gv_die import GVDie

seed = int(input("Enter seed: "))
random.seed(seed)

credits = int(input("Enter starting credits: "))

die1 = GVDie()
die2 = GVDie()

rounds = 0

while credits > 0:

    rounds = rounds + 1
    goal = -1

    die1.roll()
    die2.roll()

    roll = die1.get_value() + die2.get_value()

    if roll == 7 or roll == 11:
        credits = credits + 1

    elif roll == 3 or roll == 12:
        credits = credits - 1

    else:
        goal = roll

    while goal != -1:

        die1.roll()
        die2.roll()
```

```
        roll = die1.get_value() + die2.get_value()

        if roll == 7:
            credits = credits - 1
            goal = -1

        elif roll == goal:
            credits = credits + 1
            goal = -1

print("Game over")
print("Rounds played:", rounds)
```

13 Key Takeaways

- Objects combine state and behavior.
- A die object stores its current value.
- Methods such as `roll()` and `get_value()` define how users interact with the object.
- A random seed makes simulations repeatable.
- Larger programs become manageable when broken into loops, branches, and objects.